

Concept and architecture

Problem and user. Berlin visitors and residents need a quick way to identify a restaurant and plan an evening. Our persona is an English-speaking visitor meeting a friend for dinner. The goal is restaurant discovery and transparent planning support, without accounts or booking complexity.

Product. Search live Google Places results, confirm a restaurant, select a Berlin-local visit time and read “Tonight at a Glance.” Cards separate provider facts, unknown information and a clearly labelled crowd heuristic. Save only place identifiers in local browser storage.

Architecture. Browser → CloudFront HTTPS/WAF → Application Load Balancer → two FastAPI/Fargate tasks across two availability zones. ECR holds the portable image. Secrets Manager supplies credentials. CloudWatch records logs and alarms. Terraform describes the infrastructure. Both tasks access Google Places, Open-Meteo and optional OpenAI over HTTPS.

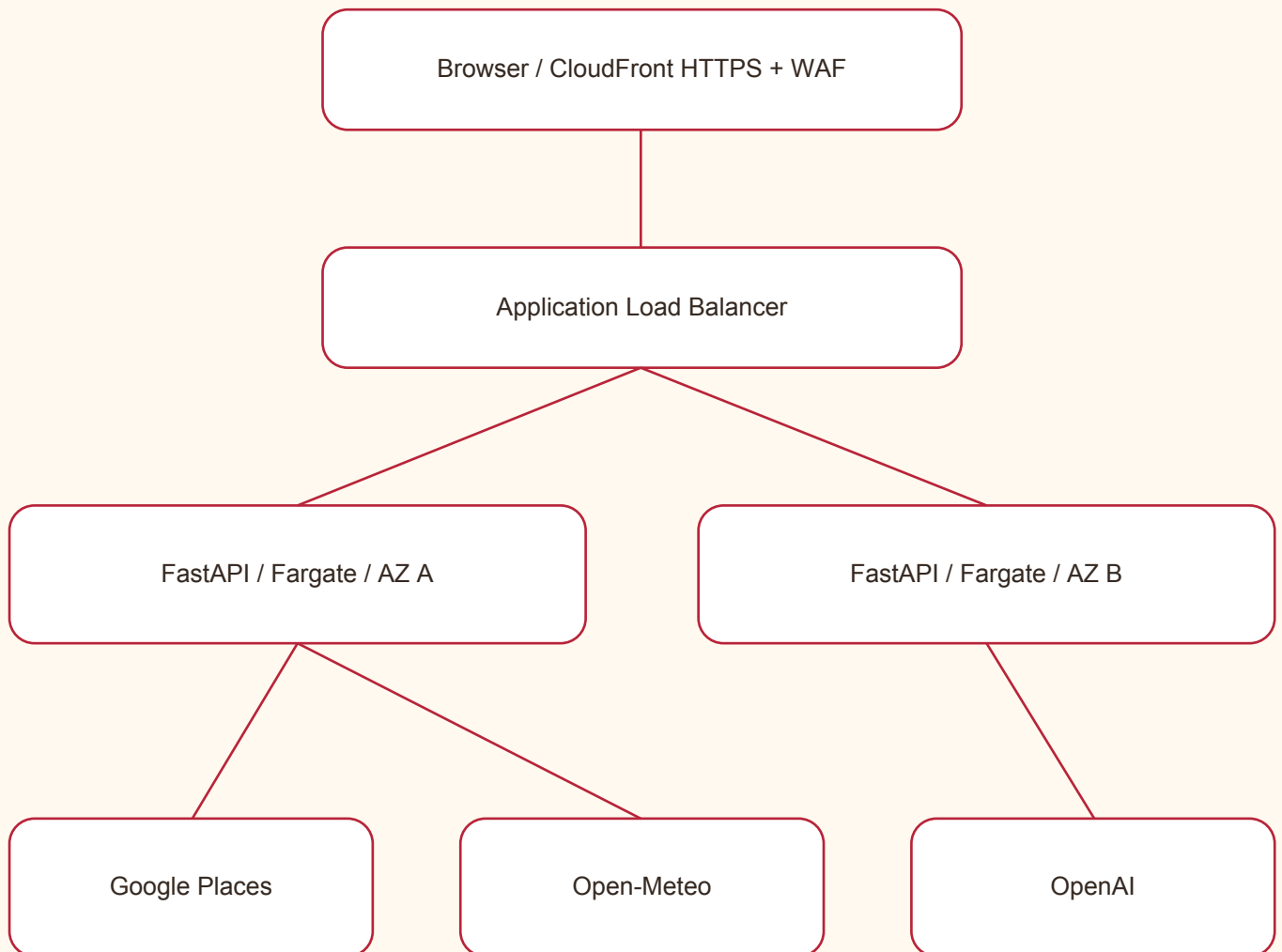
Rationale and alternatives. One Docker image runs locally or on AWS. Service adapters isolate providers. Standard ECS exposes architecture choices for the portfolio. App Runner was rejected after verifying its closure to new customers. Lambda and a separate S3 frontend remain alternatives. Omitting a database reduces cost but saved lists do not sync.

Best practices and limits. Two tasks, health checks and CPU scaling support availability. Secrets stay out of source. WAF limits traffic. Public-subnet tasks avoid NAT costs and accept inbound traffic only from the ALB. The CloudFront-to-ALB hop is HTTP in this draft; a domain and ACM certificate are needed for end-to-end TLS. Always-on resources incur cost.

Data science and honesty. Crowd labels use unvalidated time rules, not live occupancy or a trained model. English support and reservations stay unverified. Google editorial text is displayed separately and is not sent to OpenAI. Future ML requires licensed observations and evaluation.

Status. Code is drafted and basic static/scoring checks passed. All 12 automated tests, Docker runtime checks and Terraform validation passed. Real Google search passed. AWS deployment is complete. The public HTTPS homepage, real search, details and AI insights returned HTTP 200; both Fargate tasks are running. The student must review the code and institutional AI-use rules before submission.

Architecture / proposed deployment



Both tasks execute the same provider adapters.

ECR supplies the image; Secrets Manager supplies private keys.

CloudWatch collects task logs and a target-error alarm.

Terraform describes the infrastructure and scaling.

Origin hop uses HTTP in this draft; external APIs use HTTPS.

Diagram is a design, not proof of a completed AWS deployment.